

Trae 开发 Trae: 自定义模型支持 AWS 供应商

本文作者：王鸿兴，TRAE技术专家



该需求属于较为复杂的需求，按照这次的验证得到的针对复杂需求的结论：

- 能通过 AI 提升效率的：
 - 熟悉项目 & 技术方案输出；
 - 明确技术方案后的编码；
 - 问题明确（有较为明确的错误信息）的 Bugfix；
- 比较难通过 AI 提升效率的：
 - 复现难度较高的问题排查；（或者说：开发者都没头绪问题出在哪里的 bug，AI 也比较难介入）

但是纵使是能通过 AI 提升效率的环节，AI 的提效也比较难做到 100%，因为人的 Review 必不可少，此次实验中，整体耗时 7 人日（沟通 1d，熟悉 & 分析 2d，开发 1.5d，Bugfix 2.5d），如果完全无 AI 介入的情况下预计要 10 人日（沟通 1d，熟悉 & 分析 3d，开发 3d，Bugfix 3d），**整体提效 30%**，**AI 代码贡献率 95.47%**

背景

需求：当前自定义模型上云链路未支持 AWS 供应商，因为 AWS 的 converse stream 接口非通用的 openai 的 chat_completion 接口，当前实现无法兼容，需要针对 AWS 厂商做兼容接入；

思路：通过 Trae 开发 Trae 的方式来完成这个需求，看有哪些是 AI 明确可以提效，哪些是 AI 无法介入的，以此验证以下两种可行性：

- 新人通过 Trae 快速上手开发新功能（开始这个需求之前我对当前自定义模型流程不了解，可以算新人）；

- 通过 Trae 完整开发一个复杂需求（对接 AWS 的 converse stream API 的复杂度不算低，而且横跨多个系统）；

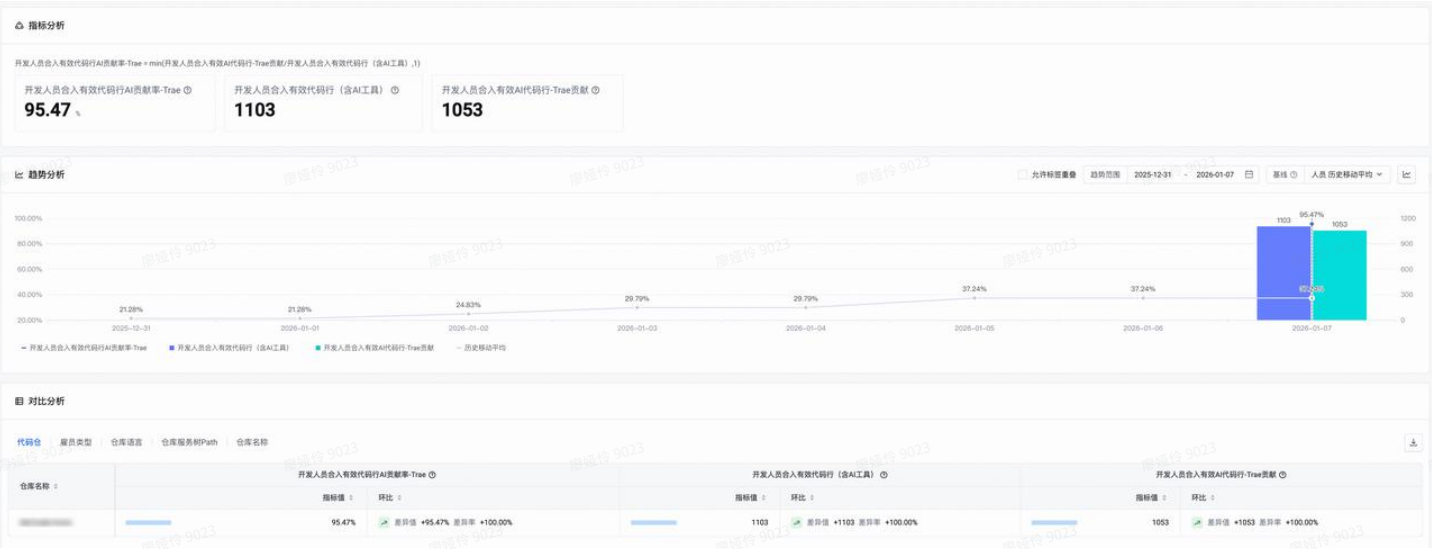
过程

按照一开始与起初负责自定义模型的同学沟通的支持思路输出了方案，但是随着对流程做了进一步分析和验证后，发现最终实现的思路与最初方案有了不少出入。

整体的时间节奏是，12.24 号开始全人力投入，1.7 验证通过并且合入 MR，整体时间经过为：

- 方案沟通：12.24（1d）
- 分析阶段：12.25 ~ 12.26（2d）
- 实施阶段：12.29 ~ 12.30（1.5d）
- Bugfix 阶段：12.30 ~ 1.7（2.5d）

AI 代码贡献率：95.47%（1053 / 1103）



分析阶段（提效 30%）

这个过程基本上就是借助 AI 在项目中输出方案思路，借助 review 这些方案思路的过程中帮助我快速熟悉自定义模型的交互流程，以及 AWS Bedrock 协议的使用规范；

虽然这个过程基本上 100% 都是使用 AI 进行分析并且给出方案，但是 review 这些方案以及确认方案是否正确的过程，也消耗了我较大的工作量，感觉提效率较为一般，粗估一下 AI 大概提效 30%；

📌 AI 在代码的实施部分提效还是比较明显，客户端内的实现有一千多行代码，基本上 90% 以上都是 AI 生成的，只是还是需要人为进行适当的介入，因为模型生成的代码容易漏功能或者产生一些冗余逻辑，如果没有及时 review 和纠正，代码容易成为屎山；

从减少的工作量来看，这个代码量如果完全人工实现，预计需要 3d 的开发工作，在 AI 辅助下只需要 1.5d（提效 50%）就完成了开发并且顺利将基本流程跑了起来（这里的时间分配大概是 0.5d 花在指导 AI 生成代码，1d 花在 Review AI 生成的代码）；

	目的	实现过程	备注	AI 贡献率
1	基于方案输出第一版代码	让模型按照前面输出的方案进行编码 	能按照方案基本上完成了第一版需求，这一版基本上完全不需要人为参与编写，但是也需要 review 一下代码；	100%
2	补充一些漏实现的问题	模型输出的代码比较容易缺斤少两，都需要人为介入 review 发现问题，然后再让模型补充漏掉的实现，包括： - 缺少对 contentBlockStart 事件的处理 - 缺少对 Function Call 的支持 	发现以下几个问题，会导致需要一定的人为介入修改： - AI 生成的代码质量比较一般，比如生成 json 会直接糊一大坨 json 代码而不是用更合理的 struct 来抽象，这个需要不停的提醒修改； - 会生成重复代码，比如已经有 convert_xxx 的函数了，但是 AI 在实现某一个需要同样逻辑的功能的时候会重新写一遍而不是直接调用 convert_xxx 函数；	90%

			生成的代码受历史的文件修改内容影响较大，比如 AI 生成了两个几乎一致但是不同命名的函数，我人肉将这两个人合并成一个后，下次 AI 进行修改的时候，容易把我的改动又重新改回去。（这一点比较烦人）；	
--	--	--	--	--

Bugfix 阶段（提效 10%）

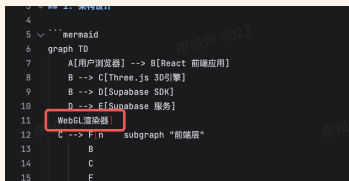


在 Bugfix 阶段主要是发现了 3 个问题：

- 前两个属于问题和错误信息都相对比较明确的 bug，所以 AI 的修复完成率都很高。
- 第三个问题是属于完全不清楚问题出在哪儿的偶现问题，而且还牵扯多个模块，AI 能介入的点就只能辅助分析一下代码是否可能有问题，剩下的只能全靠人肉去排查和定位；

整体而言提效大概 10%（问题 3 的占比太高）；

	Bug	修复过程	备注	AI 贡献率
1	当 AI 输出并发多条 Tool Call 的时候，会出现以下错误信息： 代码块 <pre>1 ValidationException: Expected toolResult blocks at messages.6.content for the following Ids: call_BjHVi0R7nItjZTtQq</pre>	先看了一下 Trace 中 AI 调用发起的 OpenAI 格式的 Messages 列表，没发现 Tool Call ID 的顺序有啥问题，那只有可能是客户端在将 OpenAI 格式的 Messages 转换成 Bedrock 格式里出了啥问题，所以先让 AI 进行分析  AI 给出了几个方案供选择，最终选择了方案 3 让模型实现，开发完成后问题得到解决；	这个问题的修复代码虽然全部都是 AI 实现，但是跟实施阶段遇到的问题一样，模型输出的代码有时候质量不太行，会选择用一些很不优雅的方案进行实现，需要人为纠正	100%

	V0Cdb4W, but found: call_l7C7J6 JQAx4nRURmn Qjjd1bY				
2	AWS 的 Tool Call 转换成 Open AI 的 toolcalls 的时候，第一条 index 较大概率为 1，会导致服务端在拼接 toolcall 的时候出现两条 toolcall 的情况	跟第一个问题的修复思路一样，先让 AI 根据现象进行分析  AI 给出了 4 个方案，考虑到 AI 推荐的方案 2 更靠谱，所以选择方案 2 进行实施，开发完成后问题得到解决： 	100%		
3	耗时最久的一个问题，大部分时间都耗在排查该问题上 发现 AI 在输出内容的时候，有非常高的概率出现内容错乱的问题，比如下面这一段就是明显错乱：  起初以为是模型幻觉，由于出现概率异常的高，深入排查才发现是 chunk 错乱；	一开始怀疑在往 Proxy 推 chunk 的时候，客户端或者服务端哪里的逻辑存在顺序漏洞，才会导致 chunk 乱序，因此让模型对代码进行了仔细的 Review  但是 AI 的分析结果都是没问题 	这个问题存在以下几个特点： - 没有明确的错误信息，也没有稳定的复现路径（只有在推理过程中有概率出现，而且是否错乱也需要人为去判断）； - 需要从整个链路去定位排查，跨多个系统（有一些还是依赖方）；	10%	

来，并且在推理过程中才可能会出现，所以只能用土法 debug 的方式，给每个 chunk 加日志的方式来排查，而且过程中 chunk 的量非常大，产生的日志量也极为庞大，还得借助 argos 上的服务端日志来联合排查，AI 能介入的可能性就更低了，因此后续这个问题的排查基本上都是纯人肉排查了。

经过两天的定位，最终定位到是客户端使用的 ttnet 网络库内部实现中存在异步的实现，会导致无法保证 websocket 的 chunk 有序，后续 ttnet 的同学进行了修复；

一些经验

- **有方案再实施（spec driven）**：避免一上来就让 AI 改代码，哪怕是一些问题相对明确的 Bugfix，尽量都让 AI 先输出方案，确认方案没问题后再执行，可以减少返工率；
- **多给案例**：避免让 AI 自由发挥，尤其是一些 sdk 的使用，AI 训练数据不一定跟得上 sdk 的更新，一些开源的 SDK 可以考虑将对应 SDK 源码下载到本地，通过软链等方式让 AI 感知到，并且结合 SDK 的源码来输出方案；
- **做好 Review 的工作**：需要多 Review AI 输出的代码，按照目前 AI 输出代码的质量，还是比较难做到放任其实现，否则 AI 容易输出一些冗余，低质量实现，如果不及时纠正，极其容易迭代成 AI 屎山；
- **你必须懂如何实现**：这个其实算是上一个经验的补充，只有对整个方案了若指掌，才能做好 Review 的工作，不然 AI 容易输出一些缺斤少两的代码

更多信息请关注 TRAE 企业版服务号 



微信搜一搜



TRAE企业版